

Contents

Audio Corpus Preparation — User Manual	1
1. Overview	1
Design principles	1
Architecture	1
2. Installation	2
3. The Audio Format and Stats panel	2
3.1 Inspection and the file table	2
3.2 Duration statistics and histogram	2
3.3 Querying and splitting the corpus	2
3.4 Format homogenisation	3
4. The Audio Slicer panel	3
4.1 Short-material workflow (interactive)	3
4.2 Long-material workflow (automated)	4
4.3 The principle of adaptive interface complexity	4
5. Output and folder structure	4
6. A typical session	5
7. Dependencies	5
8. Troubleshooting	5

Audio Corpus Preparation — User Manual

1. Overview

The Audio Corpus Preparation tool is a Max application for inspecting, homogenising, and slicing a folder of audio files before descriptor extraction. It is the first stage of the corpus-based workflow: it turns a heterogeneous folder of source material into a coherent set of analysis-ready audio slices, which the second tool (Descriptor Preprocessing) then analyses and conditions.

The tool answers two practical needs. First, it lets you *discover* the properties of a corpus — durations, sample rates, channel counts — rather than assuming them. Second, it produces sliced and format-homogenised material at the granularity later analysis requires, without ever altering the original files.

Design principles

- **Format-agnostic entry.** The tool accepts any folder of audio files regardless of internal heterogeneity, reading the sample rate, channel count, and duration of each file into a sortable table.
- **Non-destructive, copy-producing workflow.** Every operation that generates derivative material (slicing, format conversion, query splitting) writes to a dedicated subfolder and never modifies the originals. Any result can be loaded back into the inspector with a single action.

Architecture

The tool is a single Max patch with two panels working on the same corpus:

- **Audio Slicer** (left) — loads a file, computes novelty markers, displays the waveform with editable markers, and slices in one of three modes.

- **Audio Format and Stats** (right) — shows the file table, the property filter, the duration statistics with histogram, and the format-conversion controls.

It is built on **FluCoMa** (novelty detection, dataset/query objects, buffer statistics) and **EARS** (format conversion, slicing, trimming, fades, I/O). The custom waveform display (`lor.waveform.js`) is adapted from FluCoMa's `fluid.waveform~`.

2. Installation

The tool is distributed as a Max package. To install:

1. Copy the package folder into `Documents/Max 9/Packages/`.
2. Install the **FluCoMa** and **Bach/EARS** packages from the Max Package Manager if they are not already present.
3. Open the main patch.

Because Max adds the entire contents of a package to its search path, all of the tool's patches and scripts are found automatically, wherever they are placed inside the package folder. No manual path configuration is required, and the internal layout can change freely as files are added.

3. The Audio Format and Stats panel

3.1 Inspection and the file table

Drop a folder of audio files onto the drop area. The inspector reads every file and populates a sortable table with four columns: **Name**, **duration** (ms), **SR** (sample rate), and **channels**. Mixed formats are therefore immediately visible — a corpus containing both 44.1 kHz mono and 48 kHz stereo files shows both at a glance. Clicking a column header sorts the table by that property.

3.2 Duration statistics and histogram

From the file table the tool computes summary statistics of the duration distribution: the quartiles (Q1, Q2/median, Q3), the interquartile range (IQR), and the minimum and maximum. These are computed with `fluid.bufstats~` using the 25th/75th percentiles for the quartiles and the 0th/100th for the extremes.

A configurable histogram displays the duration distribution. You set the **number of classes** (bins); the histogram is drawn by a custom panel that shows the class counts together with the boundary values, so you read both the shape of the distribution and the resolution of the classification at once. This is the view that tells you whether your corpus is dominated by short grains, long files, or a mix — which in turn informs the slicing decision.

3.3 Querying and splitting the corpus

A query mechanism (built on `fluid.datasetquery~`) filters the corpus by file property. A rule is defined either by choosing a **column**, an **operator**, and a **value** through the interface, or by typing it directly into a text box — for example, isolating all files shorter than a given duration, or all stereo files.

The result of a query is itself a corpus subset, which can be inspected with the same statistical tools. From a query you can write two folders:

- the matching files to a subfolder named **selected**, and
- the complement (the files *not* matching the query) to a separate **remaining** subfolder.

A corpus can thus be split into two coherent parts in a single operation — keeping one and setting the other aside. The file paths of the selection can also be exported as a `.txt` file for later use.

3.4 Format homogenisation

Format conversion operates through EARS (`ears.format~`) and has two independent controls:

- **Sample-rate conversion** — pick a target rate from the menu (44100, 48000, 88200, 96000 Hz) or keep the original rate.
- **Channel conversion** — stereo to mono, mono duplicated to stereo, or a user-defined multichannel layout.

Either operation can be applied to the whole corpus or to the queried subset. This selective application is deliberate and distinguishes format conversion from slicing: format conversion is a per-file operation, so applying it to a subset is meaningful, whereas slicing is always applied to the whole folder. Converted files are written to their own subfolder.

4. The Audio Slicer panel

The slicing panel provides two complementary workflows, chosen according to the expected duration of the input material.

4.1 Short-material workflow (interactive)

For material where each cut matters, the slicer offers three sub-modes:

Duration mode. Slices the file into chunks of a fixed length. The markers are computed automatically from the requested chunk duration, shown on the waveform for reference, and then committed.

Number mode. Slices the file into a fixed number of equal chunks. As in Duration mode, markers are computed automatically and shown before committing.

List mode (novelty-based). Loads a single file and analyses it with FluCoMa's `fluid.buftenoveltyslice~` object to produce candidate markers at points of spectral change. The waveform and its markers are drawn in a custom graphical object where you can inspect, move, add, or remove markers by hand. Clicking the segment between two markers auditions that slice before committing, giving direct acoustic feedback alongside the visual editing. Once the markers are validated, the slices are written to a subfolder.

The novelty analysis exposes its full parameter set so it can be tuned to the material:

Parameter	Default	Description
Algorithm	0 (spectrum)	The feature on which novelty is computed
FFT size	4096	Analysis window size
Filter size	3	Smoothing of the novelty curve; larger values give fewer, coarser markers
Threshold	0.5	Novelty level above which a marker is placed; lower values give more markers

4.2 Long-material workflow (automated)

For long source files — a single recording of substantial duration, or an entire folder of long files — interactive marker editing is impractical. This workflow (`ears.split~`) takes a target chunk duration and produces equally spaced slices through the whole material, with several safeguards applied automatically:

- **Silence trimming** at slice boundaries (`ears.trim~`, threshold 0.05).
- **Short fades** on each chunk to prevent clicks (`ears.fade~`, 0.05 in / 0.05 out).
- A **minimum slice duration of 50 ms**, below which a trailing fragment is discarded (the status window reports “sample length < 50 ms, sample dropped”).
- An optional **forced output sample rate**.

Folder input is supported only in this mode, for batch processing of entire collections; no waveform is shown, since the scale of the operation makes per-file inspection pointless. (Attempting to load a folder in the short-material modes reports “folders are supported in Long mode only”).

4.3 The principle of adaptive interface complexity

The amount of control the tool exposes is matched to the granularity of the task. Short material, where each cut matters, is given fine-grained interactive control with audition. Long material, where the intent is to chunk efficiently rather than inspect each cut, is given only the few parameters that vary meaningfully at that scale.

5. Output and folder structure

Every derivative operation writes to its own clearly named subfolder and leaves the originals untouched:

- **Slices** → a subfolder of sliced audio.
- **Converted files** → a subfolder of format-converted audio.
- **Query results** → `selected` and `remaining` subfolders.

Any of these can be loaded straight back into the inspector, so you can analyse, query, or further process a result without leaving the patch or navigating the filesystem by hand. This is what makes the tool iterative rather than a single linear pass: at every stage you can step back and try a different path without losing earlier results.

6. A typical session

1. Load a folder of audio material; inspect its duration distribution and format properties.
 2. If the formats are mixed, homogenise sample rate and/or channels.
 3. Slice any long material into chunks of the desired length; load the resulting slices back into the inspector.
 4. Query the slices by property to isolate a working subset; write that subset to one subfolder and its complement to another.
 5. Keep the subset as the working corpus for descriptor extraction in Tool 2.
-

7. Dependencies

- **Max 9** (for the V8 JavaScript engine used by the custom waveform and histogram panels).
 - **FluCoMa** — `fluid.bufoveltyslice~`, `fluid.bufstats~`, `fluid.dataset~`, `fluid.datasetquery~`, `fluid.audiofilesin`, `fluid.list2buf` / `fluid.buf2list`.
 - **Bach/EARS** — `ears.format~`, `ears.split~`, `ears.trim~`, `ears.fade~`, `ears.normalize~`, `ears.read~` / `ears.write~`, `ears.info~`.
-

8. Troubleshooting

A folder won't load in Duration, Number, or List mode. Folder input is only supported in the long-material workflow. Load a single file for the interactive modes, or switch to the long-material workflow for batch processing.

Slicing produced fewer files than expected. Slices shorter than the 50 ms minimum are discarded. Check the status window for “sample dropped” messages, and reduce the target chunk length or the novelty threshold if you need finer slices.

List-mode novelty markers are too many or too few. Lower the threshold (or reduce the filter size) for more markers; raise the threshold (or increase the filter size) for fewer. Markers can also be added, moved, or removed by hand on the waveform before committing.

Statistics or histogram look wrong after loading a subfolder. Make sure the subfolder was loaded back into the inspector (not just written to disk); the statistics reflect whatever corpus is currently loaded in the file table.